



BSc (Hons) in **Information Technology Specialized in AI**
IT3012 : Intelligent Agents
Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position

(0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan') .`
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states` .
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)` . For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost` .If `current_pos == goal_pos` , return the `path_taken` . Otherwise, add `current_pos` to `reached_states` .
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\{new\}} = g_{\{current\}} + 1$, $h_{\{new\}}$ using your heuristic, and $f_{\{new\}} = g_{\{new\}} + h_{\{new\}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent` .

2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos` . Call your `astar_search` method and store the returned list of actions in `self.plan` .
5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()` . Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS prioritizes nodes based only on the cost already spent to reach the node $g(n)$.

A*search considers both the cost already spent and the estimated cost to the goal $f(n) = g(n) + h(n)$

where,

$g(n)$ = actual cost from the start node to the current node

$h(n)$ = estimated cost from the current node to the goal

$f(n)$ = total estimated cost of the solution through that node

Therefore, UCS searches based only on the path cost, while A* uses a heuristic to guide the search toward the goal.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

Manhattan Distance is:

$$h(n) = |x_1 - x_2| + |y_1 - y_2|$$

In a 4-way grid, the agent can only move up, down, left and right. It cannot move diagonally.

Therefore, the agent must make at least:

$|x_1 - x_2|$ horizontal movements

$|y_1 - y_2|$ vertical movements

So Manhattan Distance gives the minimum possible number of moves required to reach the goal when there are no obstacles. Walls can only make the actual path longer; they cannot make it shorter. Therefore, Manhattan Distance never overestimates the actual cost and is an admissible heuristic.

If the heuristic were not admissible it could overestimate the actual cost to the goal. A* might then choose a wrong path and could return a non optimal solution, so the optimality guarantee of A* would be lost.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No

If the grid allows 8 way movement , the agent can move diagonally.

For example, from (0,0) - (3,3)

Manhattan Distance gives $3 + 3 = 6$

But with diagonal movement, the agent can reach (3,3) in only 3 diagonal moves. Therefore, Manhattan Distance gives 6 while the actual minimum cost is 3. It overestimates the cost, so it is no longer admissible.

For 8-way movement where diagonal and straight moves have the same cost, we should use chebyshev distance ;

$$h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

For (0,0) to (3,3):

$$\max(3,3) = 3$$

which correctly represents the minimum number of moves.

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

Using only the distance to the closest food is weak because it ignores the other food items.

A stronger heuristic would use a minimum spanning tree connecting all remaining food items.

One possible heuristic is $h(n) = \text{distance from current position to the nearest food} + \text{MST cost of all remaining food}$

The MST connects all remaining food locations with the minimum total connection cost. This provides a stronger estimate because the agent must not only reach one food item, but eventually travel through the

locations of all remaining food. Therefore, this heuristic considers the overall distribution of the remaining food rather than focusing only on the nearest item.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING